

Leakage-Aware High-Load Anomaly Detection in Smart-Home Energy Data with Chronological Validation

Le Quoc Chung¹, Bui Doan Quang Hung¹, Nguyen Xuan Binh¹,
Thu Le¹, Tung Nhat Le¹

¹Faculty of Artificial Intelligence, FPT University, Vietnam.

Abstract

High-resolution smart-home data can support household energy monitoring, but seasonal demand, rare events, and proxy labels make anomaly evaluation vulnerable to misleading results. This paper presents a revised end-to-end pipeline for detecting unusually high one-minute household loads in the HomeC dataset. The revision identifies a target-leakage channel: *total_appliance* is an additive component of whole-house use, while the anomaly label is derived from that same use variable. The deployable models therefore exclude this feature. Labels are generated causally by comparing each observation with the mean plus three standard deviations of the preceding 30 days, excluding the current observation. Data are split chronologically into 70% training, 15% validation, and 15% test windows, and decision thresholds are calibrated only on validation data. A one-feature leakage diagnostic reaches test F1 = 0.4866, whereas the best leakage-free test F1 is 0.0839. XGBoost is selected by validation F1 = 0.1356 and obtains test ROC-AUC = 0.7771 but test F1 = 0.0000 at the frozen threshold. The paper therefore presents a reproducible correction of leakage, seasonal label bias, and threshold-transfer risk rather than claiming a deployment-ready detector.

Keywords: smart-home energy, anomaly detection, target leakage, chronological validation, time series, XGBoost, dashboard

1 Introduction

Smart meters and connected household devices make it possible to observe domestic electricity demand at minute-level resolution. These data can support dashboards, alerts, and decision support for energy saving. A high reading, however, is not automatically a fault. It may reflect weather, season, daily routine, appliance cycling, or a short high-load event that is normal in context. This makes high-load anomaly detection both attractive and methodologically fragile.

The HomeC project originally followed a common supervised-learning pattern: clean the data, define high-load anomalies using a 3-sigma rule on whole-house consumption, engineer features, train several classifiers, and show a dashboard. During revision, the experiment revealed a more important story. The feature named *total_appliance* is constructed by summing appliance channels, and those channels form a large part of the same whole-house consumption variable used to define the label. If that feature is given to a model, the model is not learning an independent warning signal. It is receiving a near-direct view of the target definition.

This paper reframes the project around that finding. Instead of presenting a high headline score, it documents a pipeline that makes leakage visible, removes the leaky channel from deployable models, defines labels using only past context, and evaluates chronologically. The result is less flattering but more useful: the remaining weather and calendar features have limited power for minute-level high-load anomaly prediction, and validation-tuned thresholds transfer poorly into the final test period. Such a negative result is still valuable because it prevents a dashboard from implying readiness that the evidence does not support.

The analysis therefore asks, in prose rather than as a separate checklist, whether the additive appliance aggregate explains the apparent performance, whether a causal local threshold gives a fairer seasonal label, and whether the remaining weather, solar, and calendar variables can support reliable prediction under chronological evaluation. These questions shape the pipeline and the interpretation of every result reported below.

The paper makes four contributions. First, it gives an end-to-end pipeline from raw or cleaned HomeC data to model artifacts and dashboard alerts. Second, it identifies and quantifies a target-leakage channel through an explicit one-feature baseline. Third, it replaces a global high-load label with a causal prior-30-day label that reduces seasonal bias. Fourth, it reports held-out results, limitations, and next steps in a form suitable for reproducing or extending the project.

2 Related Work

Energy monitoring has long studied household load signatures and appliance-level behavior. Nonintrusive appliance load monitoring showed that aggregate power measurements can reveal useful structure, but it also made clear that household energy signals mix device operation, occupant routine, and noise [1]. Modern smart-home datasets increase temporal detail, yet the interpretation problem remains: an event that looks unusual statistically may still be normal for the household.

General anomaly detection methods include density-based, isolation-based, reconstruction-based, and supervised approaches [2]. Isolation Forest is often used when explicit labels are weak or unavailable because it isolates rare observations through random partitions [3]. Supervised tree ensembles such as XGBoost and LightGBM are strong tabular learners [4,5], but they can exploit proxy variables that would not be valid in deployment. In a label generated from the target signal, this risk is especially high.

Target leakage has been described as the use of information in training that would not be legitimately available at prediction time [6]. Leakage can be accidental: an engineered feature may appear harmless even though it encodes part of the label. Time-series evaluation adds another constraint. Random splits can leak future seasonal regimes into the training set, so chronological splits and validation-only threshold selection are closer to deployment conditions. These ideas guide the revised HomeC experiment.

2 Methodology

2.1 Dataset and preprocessing

The project uses the HomeC smart-home energy dataset after local cleaning. The final table is stored as *HomeC_cleaned_final.zip* for the dashboard and as a full CSV for reproducibility. The raw time counter cannot be treated as a reliable Unix timestamp, so the pipeline reconstructs a one-minute timeline from 2016-01-01

00:00:00 to 2016-12-15 22:29:00. This assumption is recorded as a limitation rather than hidden in preprocessing.

Table 1 Summary of the processed HomeC record.

Property	Value
Date range	2016-01-01 to 2016-12-15
Sampling interval	1 minute (reconstructed)
Total household energy	7,214.00 kWh
Mean household power	0.859 kW
Peak household power	14.7146 kW
Peak timestamp	2016-07-30 16:04
Total solar generation	640.21 kWh
Local 30-day anomalies	11,140 minutes (2.21%)

Household Energy Consumption Over 2016

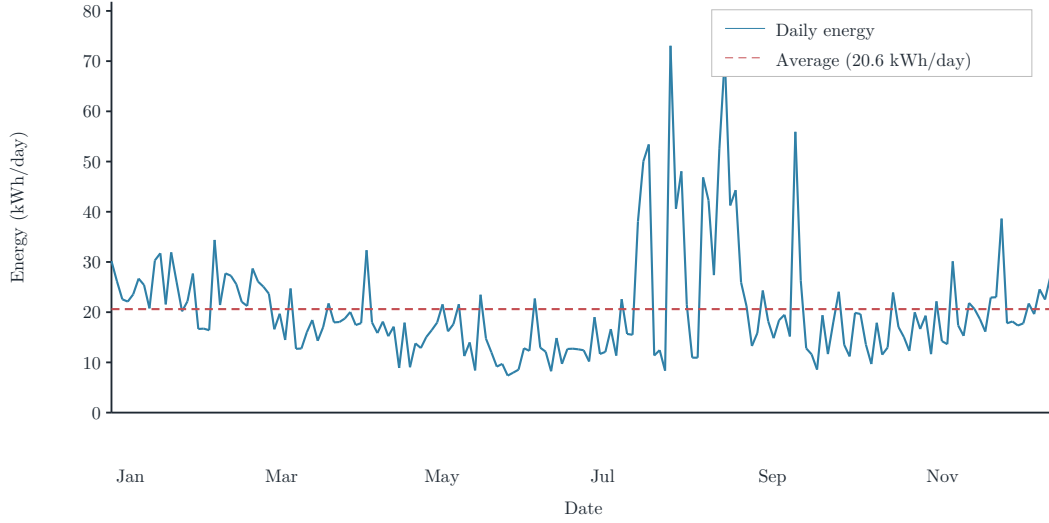


Fig. 1 Daily household energy consumption in 2016. Seasonal regimes and isolated high-demand days make a single whole-year threshold potentially misleading.

Preprocessing removes stale engineered columns before reruns, reconstructs the timeline, imputes numeric missing values with medians and text fields with modes, removes duplicate rows and columns, drops constant fields, creates calendar features, and combines appliance groups such as kitchen and furnace channels.

Power readings are converted to energy summaries by multiplying one-minute kilowatt readings by 1/60 hour. The project exports KPI JSON, top-appliance tables, weather correlations, static exploratory figures, and the cleaned ZIP consumed by the dashboard. These artifacts make the analysis repeatable and keep the dashboard independent from expensive preprocessing.

Table 2 Largest appliance groups by annual energy use.

Appliance group	Energy (kWh)	Share	Peak hour	Peak date
Furnace	1,981.96	39.41%	05:00	2016-02-14
Home office	682.69	13.58%	21:00	2016-08-09
Fridge	533.78	10.62%	20:00	2016-08-14
Barn	491.56	9.78%	16:00	2016-09-11
Wine cellar	353.88	7.04%	17:00	2016-08-14

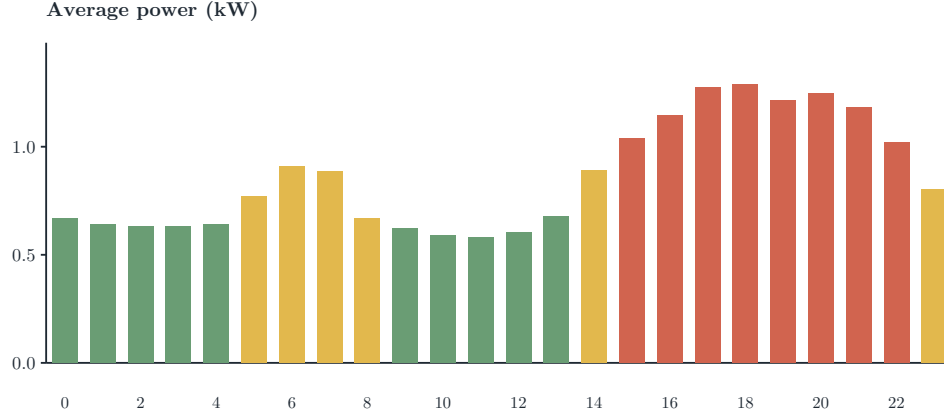


Fig. 2 Average household power by hour. The recurring daily profile provides context for alerts but does not, by itself, capture recent load dynamics.

The hourly profile has a clear evening peak, but that profile is an average over the full year. It helps describe ordinary household rhythm, while the anomaly task still requires judging whether a specific minute is high relative to its recent regime. This is why the label design later uses a rolling prior window rather than only static calendar variables.

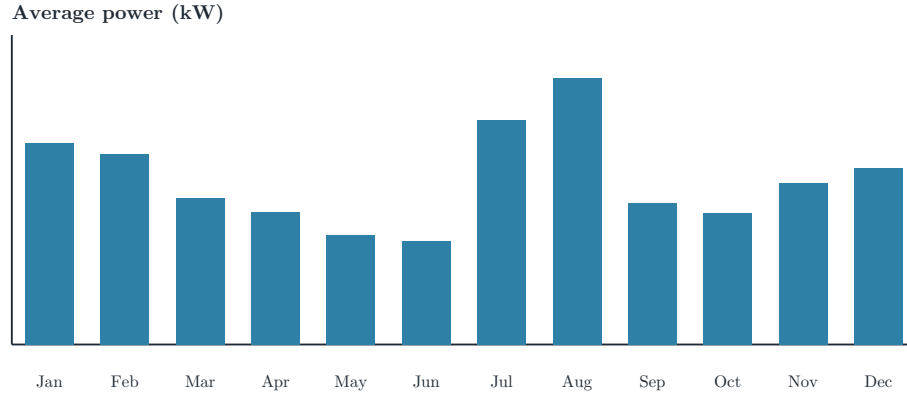


Fig. 3 Average household power by month. July and August form a sustained high-demand regime rather than isolated point noise.

The monthly profile confirms the seasonal pattern visible in the daily series. Average power rises sharply in July and August, then falls in September before increasing again near winter. A global threshold treats these regimes as if they came from one stationary distribution. A local threshold does not solve the entire problem, but it makes the comparison closer to the way an operator would inspect a household: current behavior is compared against recent behavior, not against a single full-year average.

2.2 Feature engineering and leakage control

Figure 4 shows the revised pipeline. The important design choice is that the research pipeline keeps diagnostic features separate from deployable features. *total_appliance* is still useful for a leakage test, but it is not allowed into the model that feeds the dashboard prediction form. This distinction prevents the system from learning a shortcut that would collapse once the same additive signal is unavailable or conceptually invalid at prediction time.

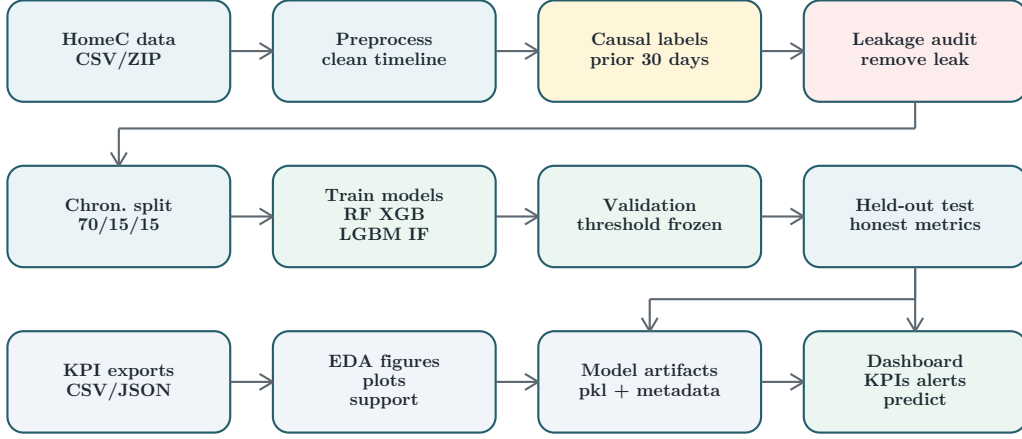


Fig. 4 Revised end-to-end pipeline with causal labels, leakage audit, chronological validation, held-out testing, and dashboard artifacts.

The deployable feature set contains seven variables: solar generation, temperature, humidity, hour, day of week, month, and weekend flag. These features are weak by themselves, but they are clean under the stated deployment scenario. The pipeline deliberately reports this weakness rather than hiding it behind a leaky predictor.

Table 3 Feature governance used to separate diagnostics from deployable prediction.

Signal group	Deployment use	Reason
Whole-house use	Label source only	Defines high-load events and is not used as a predictor.
total_appliance	Diagnostic only	Additive component of whole-house use; retained only to quantify leakage.
Solar generation	Used	Available exogenous context with clear relation to daylight and generation.
Weather	Used	Temperature and humidity are deployable, but direct correlation with use is weak.
Calendar	Used	Hour, day of week, month, and weekend represent routine and season.
Lagged load	Future work	Potentially strong signal, but not yet engineered in the current project.

2.3 Causal anomaly labels

The initial global high-load label marks an observation as anomalous when whole-house use exceeds the training mean plus three standard deviations. Calculating the threshold on training data avoids direct test leakage, but a single yearly threshold still confounds seasonal regimes. In HomeC, summer months have a different load profile, so a global threshold can classify seasonal peaks as anomalies rather than context-relative events.

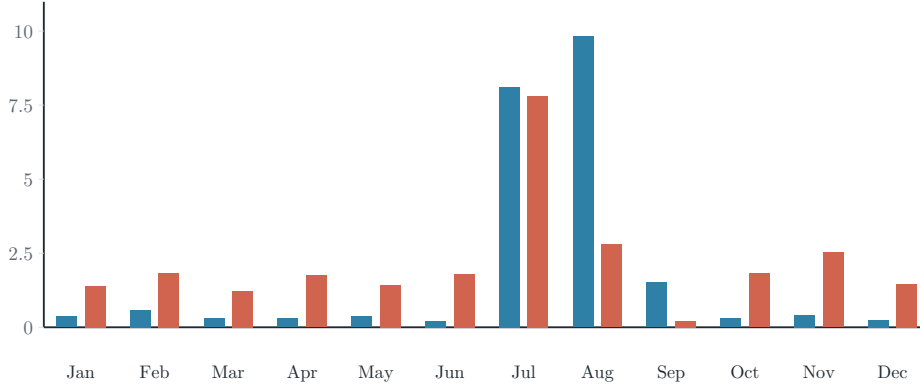
The primary label is therefore local and causal. For each time t , the pipeline calculates the mean and standard deviation of the previous 30 days only. The current row is shifted out of the rolling window, so the observation cannot contribute to its own threshold. When insufficient prior history is available, the training mean and standard deviation provide a fallback.

Primary rule: $y_t = 1$ if use_t is greater than $\text{mean}(\text{previous 30 days}) + 3 * \text{std}(\text{previous 30 days})$; otherwise $y_t = 0$.

Table 4 Anomaly-rate shift under global and local labels.

Label	Train	Validation	Test
Global train-only 3-sigma	2.527%	0.983%	0.368%
Local prior-30-day 3-sigma	2.505%	0.693%	2.355%

Anomaly rate (%) ■ Global ■ Local

**Fig. 5** Monthly anomaly rates under global and local high-load labels.

The strongest change occurs in August, where the anomaly rate drops from 9.810% under the global label to 2.789% under the local label. July remains high at 7.807%, suggesting a longer load regime that a 30-day window does not absorb immediately. This behavior is more plausible than a single global cutoff because it exposes where the proxy label still struggles.

2.4 Chronological split and threshold calibration

The cleaned records are split by time into 70% training, 15% validation, and 15% test windows. Models are fitted on the training window. Their probability or anomaly scores are evaluated on validation, and the decision threshold that maximizes validation F1 is frozen. Test metrics are then computed once on the final held-out period. This protocol is stricter than random splitting and directly reveals whether the validation threshold transfers to a later season.

Algorithm 1. Leakage-aware chronological evaluation

Input: cleaned one-minute HomeC table D , whole-house use u_t

1 Split D by time into train, validation, and test windows.

2 Build global label from train mean + 3 * train standard deviation.

3 Build local label with shifted prior-30-day rolling statistics.

4 Run one-feature diagnostic using `total_appliance` only.

5 Remove `total_appliance` from the deployable feature set.

6 Fit Logistic Regression, Random Forest, XGBoost, LightGBM, and Isolation Forest.

7 Select each model threshold on validation F1 only.

8 Select the deployable model by validation F1 in the primary experiment.

9 Report frozen-threshold test F1, ROC-AUC, precision, recall, and confusion matrix.

Output: `best_model.pkl`, `feature_columns.pkl`, `feature_defaults.json`,
`model_metadata.json`.

3 Experimental Setup

The experiment is organized into three families. The first family contains two baselines: a majority-normal predictor and a one-column threshold on `total_appliance`. The second family uses the global train-only label and compares leaky features with leakage-free features. The third family is the primary experiment, using the local prior-30-day label and the leakage-free feature set.

The supervised models are Logistic Regression, Random Forest, XGBoost, and LightGBM. Isolation Forest is included as an unsupervised comparator. Because anomalies are rare, class balancing or scale-pos-weight settings are used where

appropriate. For speed and reproducibility, tree counts are intentionally modest. The aim is not to squeeze out a leaderboard result but to compare the effect of leakage control and label design under a realistic split.

Metrics are F1, ROC-AUC, precision, and recall. F1 is used for threshold selection because anomaly detection usually needs a balance between missing events and over-alerting. ROC-AUC is still reported because it measures ranking ability independent of a single threshold. In this project that distinction matters: some models rank anomalies above normal points better than chance, but the frozen threshold does not transfer well to test.

3.1 Model configuration and calibration details

The model configurations are intentionally moderate. The goal of the experiment is to test the validity of the feature and label design, not to hide leakage behind extensive hyperparameter search. Logistic Regression provides a linear baseline, Random Forest provides a bagged tree ensemble, XGBoost and LightGBM represent gradient-boosted tree families, and Isolation Forest provides an unsupervised comparator that does not use the proxy labels during fitting.

Class imbalance is handled only within the training window. For boosted classifiers, the positive-to-negative ratio estimated from training is used to weight the minority class. Isolation Forest uses a contamination value derived from the training label rate so that its output can be compared under a similar rare-event assumption. These choices keep the validation and test windows separate from model fitting.

Threshold calibration is performed after scoring the validation window. For each model, candidate score cutoffs are swept on validation and the cutoff with the best validation F1 is retained. The selected cutoff is then applied unchanged to the test window. This protocol is stricter than reporting the best test threshold, because in a deployed dashboard the threshold would have to be chosen before future data arrive.

The final deployable artifact is selected by validation F1 within the primary leakage-free experiment. This selection rule explains why XGBoost is exported even though its frozen-threshold test F1 is zero. The paper keeps this apparent contradiction visible because it is the central lesson of the project: validation ranking, test ranking, and operational alerting are related but not identical.

Table 5 Baselines and global-label leakage experiments.

Experiment	Model/rule	Val. F1	F1	AUC	Prec.	Rec.
Majority normal	none	-	0.0000	0.5000	0.0000	0.0000
total_app. baseline	total_app. only	-	0.4866	0.9235	0.9479	0.3273
Global label + leaky features	Logistic Reg.	0.2259	0.3931	0.9020	1.0000	0.2446
Global label + leaky features	LightGBM	0.2539	0.0000	0.8747	0.0000	0.0000
Global label + leakage removed	Logistic Reg.	0.0235	0.0000	0.5992	0.0000	0.0000
Global label + leakage removed	LightGBM	0.0545	0.0000	0.6981	0.0000	0.0000

4 Results

4.1 Leakage diagnostic

The one-column *total_appliance* diagnostic is intentionally simple. It asks whether a direct additive component of whole-house use can explain the label better than the proposed models. It reaches test $F1 = 0.4866$ and $ROC-AUC = 0.9235$, with $precision = 0.9479$ and $recall = 0.3273$. This outperforms the later machine-learning configurations and confirms that the initial high performance was not reliable evidence of deployable intelligence.

When the global-label experiment still includes leaky features, Logistic Regression obtains test $F1 = 0.3931$ and $ROC-AUC = 0.9020$. After the leaky column is removed, both Logistic Regression and LightGBM fall to test $F1 = 0.0000$. The pattern is a clear leakage signature: a simple variable linked to the label carries most of the apparent predictive signal.

4.2 Primary leakage-free experiment

Table 6 Primary leakage-free local-label results.

Model	Val. F1	F1	AUC	Prec.	Rec.	Threshold
Logistic Reg.	0.0284	0.0000	0.6974	0.0000	0.0000	0.820846
Random Forest	0.0474	0.0839	0.6984	0.0515	0.2275	0.195547
XGBoost	0.1356	0.0000	0.7771	0.0000	0.0000	0.831792
LightGBM	0.0653	0.0795	0.7525	0.0600	0.1174	0.692479
Isolation Forest	0.0357	0.0316	0.4299	0.0202	0.0719	0.007471

Held-out test F1

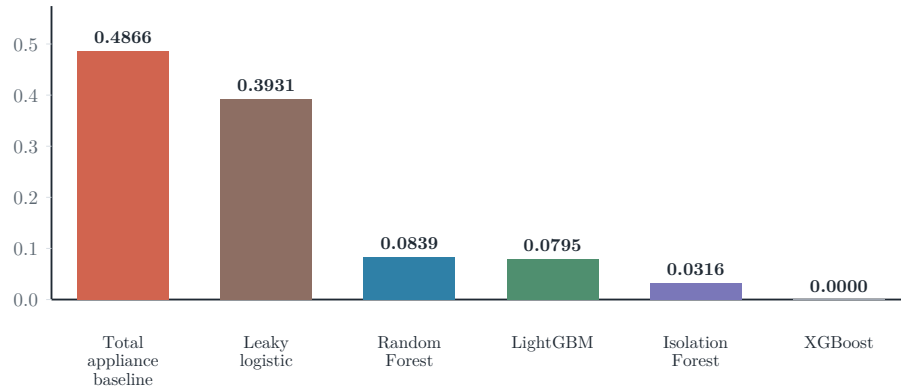


Fig. 6 Held-out F1 comparison across diagnostic and leakage-free settings.

XGBoost is selected as the deployable model because it has the highest validation F1 in the primary experiment. Its held-out ROC-AUC is 0.7771, which indicates some ranking signal. However, at the validation-selected threshold of 0.831792, it predicts no positive cases in the test window: 73,807 true negatives, 0 false positives, 1,780 false negatives, and 0 true positives. The test F1 is therefore 0.0000.

Table 7 Held-out confusion matrix for the selected XGBoost model.

	Predicted normal	Predicted anomaly
Actual normal	73,807	0
Actual anomaly	1,780	0

This result should be interpreted conservatively. It means the model and threshold are not ready to operate as an autonomous anomaly detector. The dashboard can still be used as an exploratory demonstration because it exposes metadata, alerts, and limitations, but its predictions should be treated as

experimental.

4.3 Error analysis and threshold transfer

The most important failure is not that the selected XGBoost model has no ranking signal. Its ROC-AUC of 0.7771 indicates that the score ordering is better than random on the held-out period. The failure is operational: the validation-selected threshold is too high for the test distribution, so no test observation crosses the decision boundary. This separation between ranking quality and alerting quality is common when rare-event systems are evaluated under a single frozen threshold.

Random Forest and LightGBM produce nonzero test F1 because their validation thresholds are lower. Their precision is modest, however, which means that the cost of obtaining recall is a larger number of false alerts. In a real smart-home dashboard this trade-off would need to be tuned with a user-facing alert budget rather than with F1 alone. A homeowner may prefer a small number of high-confidence alerts, while an energy auditor may accept more candidates for investigation.

The results also show why the one-feature diagnostic must remain in the paper. Without it, the low leakage-free scores might look like poor model tuning. With it, the interpretation is clearer: a shortcut feature can reconstruct the label far more easily than the legitimate exogenous variables can. The weakness of the final models is therefore evidence about the available signal, not merely an implementation accident.

For presentation, this error analysis supports a careful claim. The project demonstrates a reproducible leakage-aware pipeline and a dashboard prototype, but it does not claim an accurate autonomous detector. The honest result is still useful because it identifies the missing information required for the next revision: lagged load, rolling context features, validated event labels, and threshold calibration across multiple future windows.

5 Dashboard and Reproducibility

The dashboard is a practical part of the project because it forces the training pipeline to export machine-readable artifacts instead of leaving results inside a notebook. The application reads the cleaned ZIP, KPI summary, model metadata, feature defaults, and the selected model. It displays energy totals, peak load, anomaly counts, interactive plots, and a prediction form that uses only the leakage-free feature set.

The paper and dashboard intentionally share the same metadata. For example, the dashboard can show that the selected model is XGBoost, that the threshold is validation-selected, that *total_appliance* is excluded, and that the test F1 is 0.0000. This is not a cosmetic detail. It prevents the presentation layer from making a stronger claim than the experiment supports.

Table 8 Reproducibility and dashboard artifacts exported by the project.

Artifact	Role in reproduction or demo
experiment_results.csv	Full baseline, leakage, and primary experiment log.
model_comparison.csv	Compact comparison table for the primary local-label models.
model_metadata.json	Selected model, threshold, test metrics, confusion matrix, feature list, and limitations.
best_model.pkl	Serialized deployable model selected by validation F1.

Artifact	Role in reproduction or demo
feature_columns.pkl	Ordered leakage-free feature list required by prediction code.
feature_defaults.json	Median defaults used to initialize the dashboard prediction form.
kpi_summary.json	Data summary, label rates, monthly rates, and correlation metadata.

The project can be rebuilt by installing the listed Python requirements, running the preprocessing script, and then running the training script. The preprocessing step regenerates the cleaned data and descriptive artifacts; the training step regenerates model comparisons and serialized model files. No API key is packaged with the project, and optional explanation features fall back locally when an external key is absent.

5.1 Presentation workflow

For conference presentation, the dashboard should be introduced as the final consumer of the pipeline rather than as the proof of model accuracy. The recommended order is to begin with the data KPIs, then show the label definition, then explain the leakage baseline, and only after that show the prediction form. This order prevents the audience from interpreting the interface as a finished monitoring product before seeing the methodological constraints.

The alert view uses the same local 30-day anomaly definition as the paper. This consistency is important because it avoids a mismatch between reported model evaluation and visual alerts. The prediction form uses feature defaults exported from the training window and accepts the seven leakage-free inputs. If a presenter wants to demonstrate a high-risk case, the safer explanation is that the form illustrates model behavior under the current feature set, not that it can diagnose appliance faults.

The dashboard also gives a practical way to communicate limitations. Test F1, ROC-AUC, the frozen threshold, and the confusion matrix are stored in metadata rather than hidden in code. A human presenter can therefore show both the working software artifact and the reason why it is not yet production-ready. This is closer to an engineering research contribution than to a polished commercial claim.

6 Discussion

The experiment illustrates a common problem in data-mining projects: an impressive score can come from the structure of the label rather than from a meaningful predictor. After correction, the interpretation moves away from ranking algorithms alone and focuses on what useful signal remains after invalid information is removed. In this project, the answer is modest. Current weather and calendar variables are not enough to reconstruct minute-level high-load anomalies once the additive appliance aggregate is excluded.

The local prior-30-day label is an improvement over the global label because it compares each point with recent household context. It reduces the August seasonal spike and gives a more balanced test anomaly rate. Yet it is still a proxy. It does not know whether an event is a fault, a planned appliance cycle, an occupancy change, or a weather-driven demand period. For a real system, high-load proxy labels should be paired with event logs, occupancy signals, or expert validation.

The threshold-transfer failure is also informative. XGBoost has the strongest validation F1 and the strongest primary test AUC, but the selected operating point

is too conservative for the test period. The model may rank some positives higher than negatives, but the frozen threshold fails to cross them. Future versions should use rolling-origin validation, calibration curves, cost-aware alert thresholds, and perhaps separate thresholds by season or operating regime.

The dashboard is therefore framed as an analysis interface rather than a finished alarm product. Its value is to display KPI summaries, anomaly counts, feature defaults, model metadata, and a prediction form using the clean feature set. It also records that *total_appliance* is excluded from deployment. This makes the user-facing system consistent with the methodological claim in the paper.

7 Threats to Validity

Proxy labels. The anomaly label is a statistical high-load rule, not a verified failure label. It can identify unusual demand but cannot prove a fault or wasteful behavior.

Single-household scope. HomeC describes one household. Patterns such as appliance ownership, heating behavior, and occupancy routine may not transfer to other homes.

Reconstructed timeline. The timeline is regularized under a one-reading-per-minute assumption because the raw time counter is not used as a trusted timestamp. This supports analysis but should be documented in any reproduction.

Feature weakness. The deployment feature set intentionally excludes additive appliance information. Without lagged load, rolling statistics, or verified device states, predictive power is limited.

Operating threshold. Validation-selected thresholds may not be stable across seasonal regimes. The test result shows exactly this risk.

7.1 Future work

Add lagged whole-house load features at 5-minute, 30-minute, 1-hour, and 24-hour horizons, while ensuring they are available at prediction time.

Add rolling features such as prior-window mean, standard deviation, maximum, slope, and recent exceedance counts.

Evaluate rolling-origin validation instead of a single chronological split, so threshold stability can be measured across multiple future windows.

Collect or annotate true household events, including appliance faults, maintenance, occupancy changes, and planned high-use periods.

Test the pipeline on multiple homes to separate household-specific habits from general anomaly behavior.

Expose calibration diagnostics in the dashboard so users can see when a model ranks events but fails at its current threshold.

8 Conclusion

This paper presents a revised HomeC smart-home anomaly-detection pipeline that prioritizes methodological validity. The main correction is the identification and removal of *total_appliance* from deployable models because it is an additive component of the whole-house signal used to form the high-load label. The revised pipeline adds train-only global labels, causal prior-30-day local labels, chronological

70/15/15 evaluation, validation-only threshold selection, leakage baselines, model metadata, and a dashboard artifact path.

The final numbers are deliberately honest. A one-feature leaky diagnostic reaches test $F1 = 0.4866$, while the best leakage-free test $F1$ is only 0.0839 . XGBoost is selected by validation $F1$ but reaches test $F1 = 0.0000$ at the frozen threshold despite $ROC-AUC = 0.7771$. These results show that the current system is not a deployment-ready anomaly detector. Its contribution is a cleaner experimental foundation and a transparent warning against leakage-driven claims in smart-home energy analytics.

References

- [1] Hart, G. W.: Nonintrusive appliance load monitoring. *Proceedings of the IEEE* 80(12), 1870-1891 (1992).
- [2] Chandola, V., Banerjee, A., Kumar, V.: Anomaly detection: A survey. *ACM Computing Surveys* 41(3), 1-58 (2009).
- [3] Liu, F. T., Ting, K. M., Zhou, Z.-H.: Isolation forest. In: *IEEE International Conference on Data Mining*, pp. 413-422 (2008).
- [4] Chen, T., Guestrin, C.: XGBoost: A scalable tree boosting system. In: *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785-794 (2016).
- [5] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., Liu, T.-Y.: LightGBM: A highly efficient gradient boosting decision tree. In: *Advances in Neural Information Processing Systems*, pp. 3146-3154 (2017).
- [6] Kaufman, S., Rosset, S., Perlich, C., Stitelman, O.: Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data* 6(4), 1-21 (2012).
- [7] Kohavi, R.: A study of cross-validation and bootstrap for accuracy estimation and model selection. In: *International Joint Conference on Artificial Intelligence*, pp. 1137-1143 (1995).
- [8] Powers, D. M. W.: Evaluation: From precision, recall and F-measure to ROC, informedness, markedness and correlation. *Journal of Machine Learning Technologies* 2(1), 37-63 (2011).
- [9] Breiman, L.: Random forests. *Machine Learning* 45(1), 5-32 (2001).
- [10] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., Duchesnay, E.: Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research* 12, 2825-2830 (2011).
- [11] Kolter, J. Z., Johnson, M. J.: REDD: A public data set for energy disaggregation research. In: *Workshop on Data Mining Applications in Sustainability*, pp. 1-6 (2011).
- [12] Kelly, J., Knottenbelt, W.: The UK-DALE dataset, domestic appliance-level electricity demand and whole-house demand from five UK homes. *Scientific Data* 2, 150007 (2015).